RISC-V com Coprocessador de Multiplicação Matricial e Controlador DMA

Projeto de front-end RTL em Verilog/SystemVerilog e síntese lógica

1. Objetivo

Projetar, implementar, verificar e sintetizar um sistema digital composto por um processador RISC-V, um controlador de DMA e um coprocessador para multiplicação matricial baseado em *systolic array* parametrizável. O processador deverá apenas configurar e iniciar a operação; a movimentação de dados e a execução da multiplicação deverão ocorrer sem intervenção direta da CPU.

2. Requisitos de arquitetura mínima do sistema

- Processador RISC-V RV32I implementado em RTL.
- Interface de controle mapeada em memória para configuração do coprocessador.
- Controlador de DMA responsável pelas transferências entre a memória principal e as memórias/buffers locais do acelerador.
- Coprocessador de multiplicação matricial baseado em systolic array.
- Memória de dados compartilhada pelo processador e pelo subsistema DMA/acelerador.
- Sinalização de término por registrador de status e, preferencialmente, interrupção.

3. Operação requerida

O sistema deverá calcular C = A × B (multiplicação matricial algébrica). A CPU deverá configurar os endereços das matrizes, suas dimensões e iniciar a operação. Após o comando de início, o controlador de DMA deverá realizar autonomamente as transferências necessárias, alimentar o coprocessador e armazenar o resultado na memória.

- A CPU não poderá realizar, por software, as transferências elemento a elemento durante a operação acelerada.
- O DMA deverá possuir lógica de controle própria para leitura e escrita em memória.
- O resultado final deverá ser idêntico a uma implementação de referência executada em software.

4. Systolic array parametrizável

O coprocessador deverá usar uma malha sistólica de unidades MAC (multiply-accumulate). O tamanho da malha deverá ser definido por parâmetro de elaboração, permitindo sintetizar diferentes configurações, por exemplo 2×2, 4×4 e 8×8.

- A dimensão do systolic array deverá ser controlada por parâmetro RTL (por exemplo, ARRAY_SIZE).
- As unidades de processamento deverão operar de forma concorrente e reutilizar os dados que atravessam a malha.
- A largura dos operandos e do acumulador deverá ser explicitamente definida e justificada.
- A implementação deverá funcionar corretamente para matrizes maiores que o tamanho físico do systolic array, utilizando particionamento em blocos (tiling) quando necessário.

5. Requisitos do DMA

- Transferir blocos das matrizes A e B da memória principal para os buffers locais do acelerador.
- Transferir os resultados do acelerador para a matriz C na memória principal.
- Operar independentemente do processador após a configuração inicial.
- Disponibilizar, no mínimo, registradores de endereço-base, tamanho/dimensões, controle e status.
- Coordenar as transferências com o coprocessador por sinais de requisição, pronto/ocupado e conclusão.

6. Parametrização e síntese

O projeto deverá permitir a síntese de pelo menos três configurações diferentes do systolic array. Para cada configuração, o grupo deverá apresentar resultados de síntese contendo, no mínimo:

- área ou quantidade de recursos utilizados;
- frequência máxima estimada;
- número de ciclos para a multiplicação de uma matriz de teste;
- comparação de desempenho entre as configurações.

7. Verificação

- Criar ambiente de verificação em UVM para validar o processador, o DMA e o acelerador integrados.
- Testar diferentes dimensões de matrizes e diferentes valores de dados.
- Comparar automaticamente a matriz C produzida pelo hardware com um modelo de referência.
- Demonstrar por waveform pelo menos uma operação completa: configuração, DMA, processamento no systolic array e escrita do resultado.

8. Entregas parciais

O projeto deverá ser desenvolvido com entregas quinzenais de resultados parciais, totalizando 5 marcos de entrega:

a. Entrega 1 (Semana 4): Planejamento, Microarquitetura e Modelo de Referência

i. **O objetivo desta etapa é garantir que o grupo compreendeu o escopo e definiu como os módulos vão se comunicar antes de escrever o RTL.**

ii. **Repositório GitHub: Criação do repositório, definição da estrutura de diretórios e primeiro commit.**

iii. **Especificação (PDF ou README.md): Diagrama de blocos detalhado do sistema (Top-Level). Definição do mapa de memória (endereços base das matrizes e dos registradores do DMA/Acelerador) e especificação dos sinais de interface entre CPU, DMA e Acelerador.**

iv. **Modelo de Referência (Golden Model): Código em software (C, C++ ou Python) que realiza a multiplicação matricial por blocos (*tiling*). Esse modelo será a base para o ambiente de verificação mais à frente.**

b. Entrega 2 (Semana 6): RTL do Acelerador (Systolic Array) e Testes Unitários

i. **Código RTL (Acelerador): Implementação em Verilog/SystemVerilog da Unidade MAC e da malha sistólica parametrizável.**

ii. **Testbench Unitário: Um *testbench* focado apenas no acelerador. O grupo deve injetar os dados manualmente (simulando o comportamento do DMA) para provar que a malha processa os dados e acumula os resultados corretamente para pelo menos dois tamanhos de array (ex: 2x2 e 4x4).**

c. Entrega 3 (Semana 7): RTL do DMA e Memória Compartilhada

i. **O foco muda para a infraestrutura de movimentação de dados, que costuma ser o maior gargalo de implementação.**

ii. **Código RTL (DMA e Memória): Implementação das máquinas de estado do DMA (estado de leitura de A e B, estado de escrita de C, controle de contadores/endereços) e da memória de dados compartilhada.**

iii. **Testbench Unitário (DMA + Memória + Acelerador): Integração inicial sem a CPU. O *testbench* atua como a CPU, configurando os registradores do DMA por *bus functional models* (tarefas de escrita) e liberando o DMA para trabalhar. O objetivo é provar que o DMA consegue ler a memória, alimentar o *systolic array* construído na Entrega 2, e salvar o resultado de volta.**

d. Entrega 4 (Semana 10): Integração Top-Level, Firmware e Verificação Automatizada

i. **Código RTL Completo: Módulo Top-Level instanciando o RISC-V RV32I, a memória, o DMA e o Acelerador.**

ii. **Firmware: Código fonte e binário do programa (em C ou Assembly RISC-V) que será carregado na memória de instruções para configurar os registradores e disparar o sistema.**

iii. **UVM Integrado: O ambiente de verificação final que carrega parâmetros e estímulos e compara com o resultado do Modelo de Referência.**

9. Cronograma

Atividade	**Prazo**
Orientação Inicial	09/10/2026
Entrega Parcial 1	01/11/2026
Entrega Parcial 2	14/11/2026
Entrega Parcial 3	22/11/2026
Entrega Parcial 4	13/12/2026

Entrega final e Apresentação	18/12/20026 e 19/12/2026

Atenção: O cronograma poderá ser alterado.

A data de apresentação e entrega final não poderá ser adiada.

10. Entrega Final

O grupo deverá organizar e entregar os seguintes artefatos ao final do projeto em um respositório github.com:

a. Códigos-Fonte (Hardware e Software) e Ambiente de Simulação

i. **Código RTL: Conjunto completo de arquivos de design em Verilog/SystemVerilog, organizados hierarquicamente em diretórios (CPU, DMA, Acelerador e Integração do Top-Level). A parametrização do tamanho da malha sistólica deve estar claramente exposta no código.**

ii. **Código de Software (Firmware): O programa (em C ou Assembly do RISC-V) utilizado para configurar os endereços de memória, parametrizar as dimensões das matrizes e dar o comando de início ao acelerador.**

iii. **Ambiente de Verificação: O *testbench autochecking* completo, incluindo o modelo de referência em software (golden model) responsável por validar automaticamente a matriz resultante.**

iv. **Scripts de Automação: Arquivos (Makefiles, scripts TCL de simulação, etc.) necessários para que o avaliador consiga compilar e rodar a simulação sem intervenção manual.**

b. Arquivos de Síntese Lógica (Logs e Reports)

i. **Para comprovar o suporte à parametrização, o grupo deve entregar os relatórios brutos gerados pela ferramenta de síntese (reports de área, *timing* e utilização de recursos).**

ii. **Devem ser entregues os diretórios ou arquivos de report referentes a pelo menos 3 (três) configurações físicas distintas do *systolic array*.**

c. Relatório Técnico Consolidado (um único documento em formato PDF); conteúdo Exigido:

i. **Descrição da arquitetura implementada, detalhando a máquina de estados do DMA e o protocolo de comunicação adotado entre CPU, DMA e o coprocessador.**
ii. **Análise das 3 Configurações: Apresentação dos dados de síntese extraídos para cada uma das três configurações de malha sistólica geradas.**
iii. **Comparativo de Desempenho (Trade-offs): Uma sessão analítica cruzando os dados das três configurações, comparando o consumo de área, a frequência máxima de operação alcançada e a latência (número de ciclos de clock reais para multiplicar uma mesma matriz de teste).**
iv. **Conclusões sobre o compromisso entre o aumento do paralelismo na malha sistólica versus o custo em área e os eventuais gargalos de temporização.**

d. Apresentação e Demonstração Prática

i. **Material Visual: Arquivo com os slides utilizados na defesa do projeto.**
ii. **Demonstração Funcional (Waveform): Apresentação ao vivo das formas de onda comprovando o funcionamento integrado.**

11. Critérios de avaliação

Os critérios de avaliação serão baseados no cumprimento das entregas parciais, na corretude e completude dos entregáveis. Será avaliada não apenas a correção funcional, mas principalmente a qualidade da arquitetura RTL, a autonomia do DMA, a correta utilização do systolic array, a parametrização, a capacidade de síntese e a análise quantitativa dos compromissos entre paralelismo, área e desempenho.